INSTITUTO FEDERAL
Sul-rio-grandense
Câmpus
Charqueadas

EDUCAÇÃO PÚBLICA 100% GRATUITA

Programação Web II

Orientação a Objetos em PHP

Prof. Fábio Luís da Silva Santos
fabiosantos@ifsul.edu.br

Orientação a Objetos em PHP

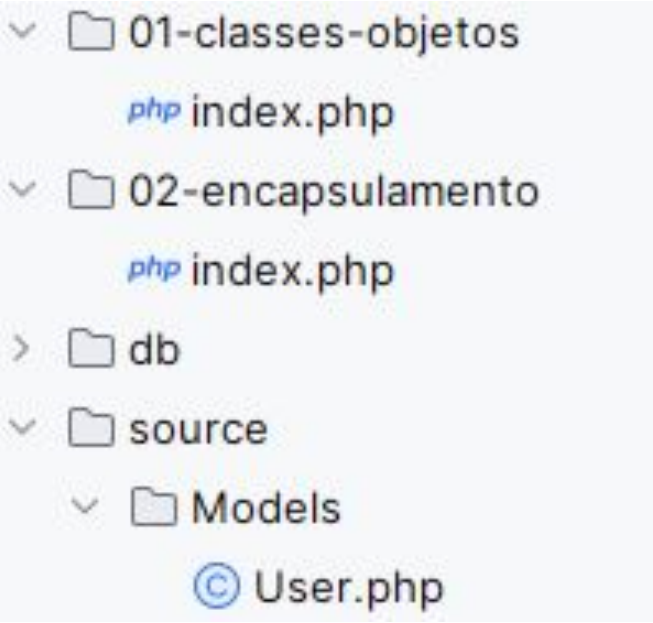
- Definição de Classes e Objetos
- Encapsulamento
- Namespaces
- Autoload
- Herança
- Polimorfismo
- Associação
- Agregação

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Definição de Classes e Objetos

Disposição das pastas e dos arquivos:



EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Definição de Classes e Objetos

source/Models/User.php

```php
<?php

class User
{
    public ?string $name;
    public ?string $email;

   public function showUser ()
   {
      echo "Olá {$this->name} seu e-mail é {$this->email}";
   }
}
```

PSR-1 - Basic Coding Standard

- Nomes de Classes devem ser declarados em StudlyCaps:
  - Por exemplo, MyClass, MyOtherClass.
- Nomes de Métodos devem ser declarados em camelCase:
  - Por exemplo, myMethod, otherMethod.
- Nomes de Métodos e Propriedades devem ser em inglês ou em um padrão internacional

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Definição de Classes e Objetos

01-classes-objetos/index.php

```php
// Inclui o script da classe em index.php
require __DIR__ . "/../source/Models/User.php";

// permite instanciar a Classe User
use Source\Models\User;

// Instancia um objeto chamado $user a partir da classe User
$user = new User();
var_dump($user);

// Acessa os atributos
$user->name = "Fábio";
$user->email = "fabiosantos@ifsul.edu.br";
var_dump($user);

// Imprime o conteúdo dos atributos
echo "Nome: {$user->name} Email {$user->email}";
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Encapsulamento

O encapsulamento é um conceito fundamental da POO. Ele permite controlar o acesso a atributos e métodos de um objeto, protegendo os dados e facilitando a manutenção do código.

- Vantagens
  - Protege os dados de alterações indesejadas
  - Mantém a integridade do objeto
  - Promove a modularidade
  - Facilita a manutenção e evolução do código
  - Ajuda a prevenir erros decorrentes de acessos incorretos aos dados
  - Permite alterar a implementação interna de um objeto sem afetar outras partes do sistema

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Encapsulamento

```
class User
{
    private ?string $name;
    private ?string $email;

    // Definição da Construtora
    public function __construct(string $name = null, string $email = null)
    {
        $this->name = $name;
        $this->email = $email;
    }
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Encapsulamento

```
    // Getters e Setters
    public function getName(): ?string
    {
      return $this->name;
    }

    public function setName(string $name): void
    {
      $this->name = $name;
    }

    public function getEmail(): ?string
    {
        return $this->email;
    }
    public function setEmail(string $email): void
    {
        $this->email = $email;
    }
}
```

Mesmo sendo opcional é importante definirmos os tipos dos métodos getters e setters:

- **getters** são definidos conforme o tipo do atributo que está sendo retornado. O ponto de interrogação (?) significa que é possível um retorno nulo;
- **setters** são sempre void, pois não retornam um valor.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Encapsulamento

02-encapsulamento/index.php

```
/** index.php */
/** Chama o arquivo da classe User*/
require __DIR__ . "/../source/Models/User.php";

/** instancia o objeto $user a partir da classe User */
$user = new User();
var_dump($user);

$user = new User("Fábio","fabiosantos@ifsul.edu.br");
var_dump($user);

echo "<p>Nome: {$user->getName()} - Email: {$user->getEmail()}</p>";

$user->setName("João");
$user->setEmail("joao@ifsul.edu.br");

echo "<p>Nome: {$user->getName()} - Email: {$user->getEmail()}</p>";
var_dump(get_class_methods($user)); // traz todos os métodos da classe
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Exercício

Complete a classe User com os seguintes atributos

- $id (int);
- $idType (int);
- $name (string);
- $email (string);
- $password (string);
- $photo (string).

Não esqueça de atualizar o método *__construct()* e dos *getters* e *setters*!

Orientação a Objetos em PHP

Hora de *Codar* INF-3AM

Exercício 01

Classe Product disponível em
https://github.com/sorfabiosantos/inf-3am-26/tree/main/exercicios/exercicio01

Exercício 02

Classe Order disponível em
https://github.com/sorfabiosantos/inf-3am-26/tree/main/exercicios/exercicio02

Exercício 03
Classe Employee disponível em:
https://github.com/sorfabiosantos/inf-3am-26/tree/main/exercicios/exercicio03

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Hora de *Codar* INF-3AT

Exercício 01

Classe Product disponível em
https://github.com/sorfabiosantos/inf-3at-26/tree/main/exercicios/exercicio01

Exercício 02

Classe Order disponível em
https://github.com/sorfabiosantos/inf-3at-26/tree/main/exercicios/exercicio02

Exercício 03
Classe Employee disponível em:
https://github.com/sorfabiosantos/inf-3at-26/tree/main/exercicios/exercicio03

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

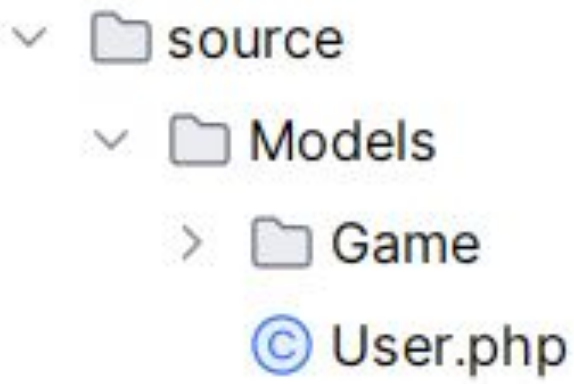
Namespaces

Em PHP, um **namespace** é uma maneira de encapsular um grupo de classes, funções e constantes em um "espaço de nomes" separado. Isso ajuda a evitar conflitos de nome entre diferentes partes de um projeto ou entre projetos diferentes.

Evita possíveis conflitos de nomes de elementos entre Bibliotecas diferentes.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Criando nossa primeira ***namespace***

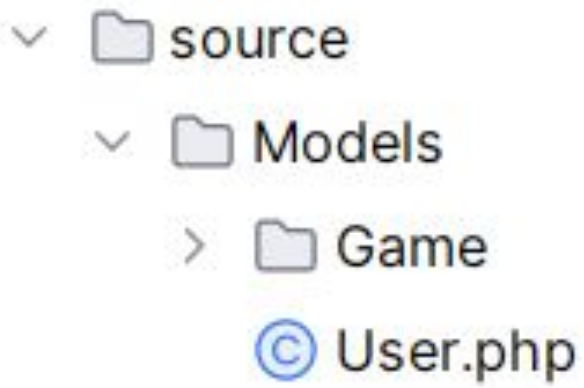


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

Agora conforme as PSR o nome do script que conterá a classe deve conter o mesmo nome da classe.



EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

\source\Models\Users.php

```php
namespace source\Models;

class User
{
    private ?string $name;
    private ?string $email;
    public function __construct(?string $name = null, ?string $email = null)
    {
        $this->name = $name;
        $this->email = $email;
    }
}
```

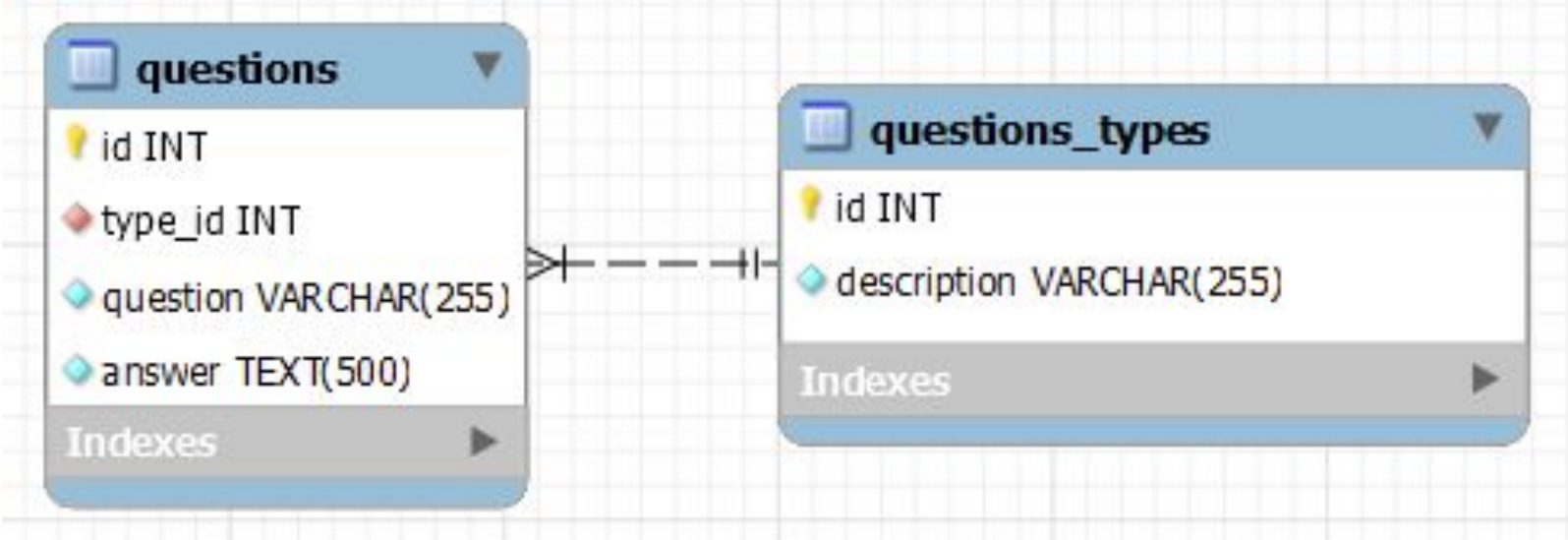
namespace: garante o desenvolvimento da aplicação sem conflitos em caso de uso de componentes de terceiros. Segundo a PSR-4 não se pode criar classes sem namespace (domínio)

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

Exemplo: o site da MOCITEC necessita de um sistema de FAQ (Frequently Asked Questions). As perguntas dos usuários são divididas em tipos, por exemplo, Inscrições, Sobre o Evento, Organização, etc… Segue o modelo com as tabelas aderentes a essa realidade.
INF-3AM: https://github.com/sorfabiosantos/inf-3am-26/tree/main/exercicios/exercicio04
INF-3AT: https://github.com/sorfabiosantos/inf-3at-26/tree/main/exercicios/exercicio04

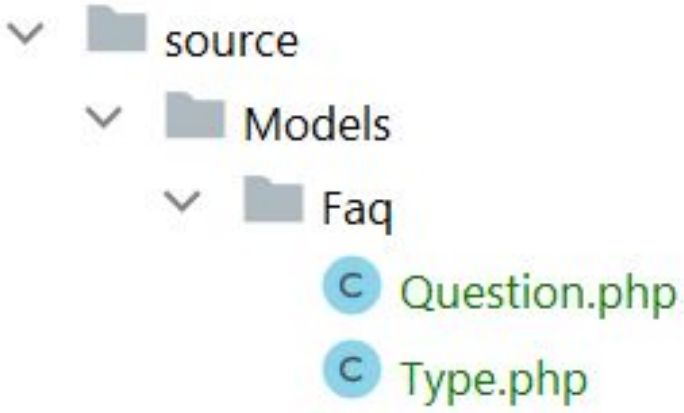


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

Vamos aproveitar esse exemplo para definirmos uma pasta nova dentro da “source\Models”, a “Faq”. Dentro da “Faq”, definiremos as classes “Question.php” e “Type.php”:



EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

source/Models/Faq/Question.php

```php
<?php

namespace Source\Models\Faq;

class Question
{
    private ?int $id;
    private ?int $idType;
    private ?string $question;
    private ?string $answer;
```

Pastas “Fornecedor”

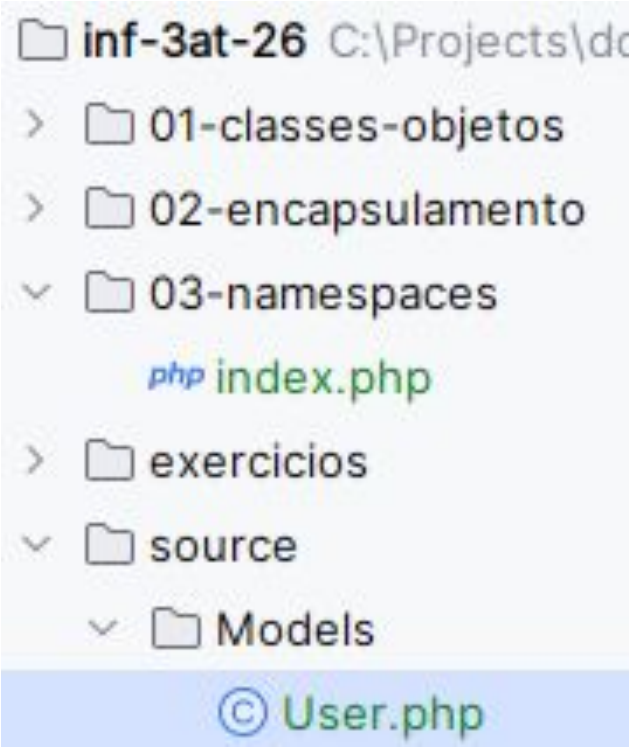
`namespace`: garante o desenvolvimento da aplicação sem conflitos em caso de uso de componentes de terceiros. Segundo a PSR-4 não se pode criar classes sem namespace (domínio)

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

Utilizando uma classe de uma namespace




```php
<?php
// o require depois não será mais necessário
require __DIR__ . "/../source/Models/User.php";

use source\Models\User;

$user = new User();
var_dump($user);
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

source/Models/Faq/Type.php

```php
<?php

namespace Source\Models\Faq;

class Type
{
    private ?int $id;
    private ?string $description;
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Namespaces

03-namespaces/index.php

Agora temos duas formas para chamarmos as classes para o “index.php”.

```
require __DIR__ . "/../source/Models/Faq/Question.php";
require __DIR__ . "/../source/Models/Faq/Type.php";

$question = new \Source\Models\Faq\Question();

var_dump($question);
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

03-namespaces/index.php

Agora temos duas formas para chamarmos as classes para o “index.php”.

```
require __DIR__ . "/../source/Models/Faq/Question.php";
require __DIR__ . "/../source/Models/Faq/Type.php";

use Source\Models\Faq\Question;
$question = new Question();

var_dump($question);
```

03-namespaces/index.php

Agora seria possível termos outra classe chamada “Type.php”, desde que ela esteja definida em outra namespace.

Orientação a Objetos em PHP

Autoload

Quando o sistema envolve muitas classes, é interessante automatizar a inclusão dos arquivos que implementam essas classes. O “autoload.php” realiza a inclusão do(s) arquivo(s) de uma/várias classe(s) quando essa(s) é/são evocada(s) de forma automática.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Autoload

/source/autoload.php

Veremos esse mesmo recurso mais tarde com a utilização do Composer

```php
<?php
spl_autoload_register(function ($class) {
    $prefix = "Source\\";
    $baseDir = __DIR__ . "/";
    $len = strlen($prefix);

    if (strncmp($prefix, $class, $len) !== 0) {
        return;
    }

    $relativeClass = substr($class, $len);
    $file = $baseDir . str_replace("\\", "/", $relativeClass) . ".php";
    if (file_exists($file)) {
        require $file;
    }
});
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Autoload

04-autoload/index.php

require apenas do autoload.php para chamar as demais classes.

```
require __DIR__ . "/../source/autoload.php";

use Source\Models\Faq\Question;
use Source\Models\Faq\Type;

$question = new Question();
$type = new Type();

var_dump($question, $type);
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Herança

Herança é um conceito fundamental da POO. Ela permite que uma classe herde atributos e comportamentos de outra classe, facilitando a criação de novas classes.

- Vantagens
  - Reutilização de código
  - Facilita a manutenção
  - Cria uma hierarquia lógica entre objetos
  - Permite a criação de sistemas mais estruturados, flexíveis e fáceis de manter
  - Deixa o código mais limpo

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Herança

```
class User
{
   protected ?int $id;
   protected ?string $name;
   protected ?string $email;

   public function __construct(?int $id = null, ?string $name = null, ?string $email = null)
   {
       $this->id = $id;
       $this->name = $name;
       $this->email = $email;
   }
}
class Employee extends User
{
   private ?float $salary;
   private ?string $department;
   public function construct(?int $id = null, ?string $name = null, ?string $email = null,
                             ? int $salary = null, ?int $department = null)
   {
       parent::__construct($id, $name, $email);
       $this->salary = $salary;
       $this->department = $department;
   }
}
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

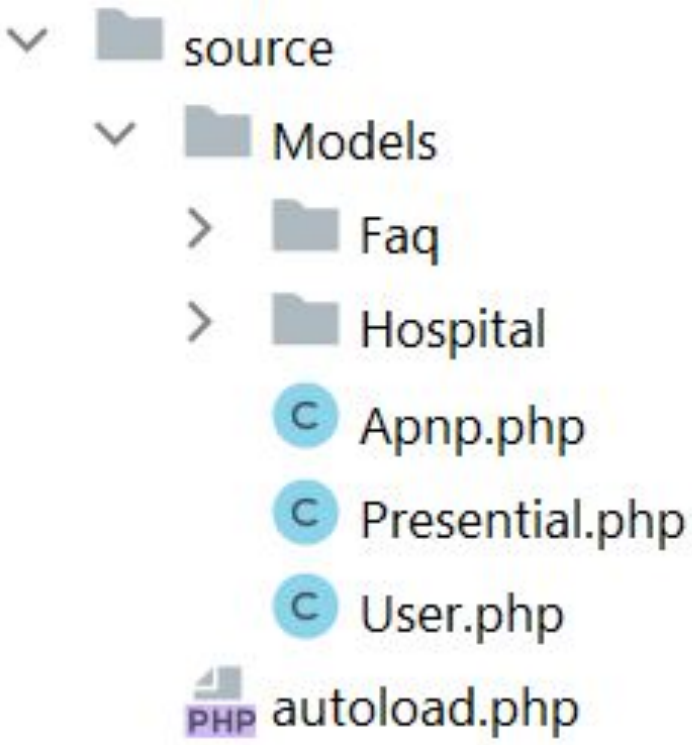
Herança

Em uma namespace chamada “Source\Models\Hospital” crie as classes para a seguinte realidade: em um Hospital é necessário manter os registros dos médicos (doctors) e pacientes (patients). O pacientes possuem em comum Id, Nome, Email e Senha (Usuários), Data Nascimento e Prontuário (texto com o histórico clínico). Já os médicos, além do ID, Nome, Email e Senha (Usuários) precisam do Registro no CRM e sua especialidade (Cardiologista, Clínico Geral, etc). Os pacientes, além do Nome e Email, precisam das informações do Prontuário e Data de Nascimento.
Utilizando Herança, crie as classes para representar essa realidade. Utilize a classe User como mãe de Paciente e Médico.

INF-3AT: https://github.com/sorfabiosantos/inf-3at-26/tree/main/hospital
INF-3AM: https://github.com/sorfabiosantos/inf-3am-26/tree/main/hospital

EDUCAÇÃO PÚBLICA 100% GRATUITA

05-heranca



EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Polimorfismo

É um princípio da POO que permite que objetos de classes diferentes respondam à mesma mensagem (método) de maneiras diferentes. Do grego "muitas formas", ele utiliza herança e sobrescrita de métodos para que uma única interface opere em diferentes tipos de objetos, tornando o código mais flexível, genérico e reaproveitável. Um mesmo nome de método pode apresentar comportamentos diferentes, dependendo da classe que o invoca.

Classes filhas redefinem métodos de classes pai.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Polimorfismo

```php
class Animal
{
    public function makeSound(): string
    {
        return "Som genérico";
    }
}

class Dog extends Animal
{
    public function makeSound(): string
    {
        echo parent::makeSound() . "<br>";
        return "Au Au!";
    }
}

class Cat extends Animal
{
    public function makeSound(): string
    {
        return "Miau!";
    }
}
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Polimorfismo

```
class Employee extends User
{
   protected ?float $hoursWorked;
   protected ?float $hourValue;

   protected function calculateSalary(): float
    {
    return $this->hoursWorked * $this->hourValue;
    }
}

class Seller extends Employee
{
   protected ?float $totalSale;

   public function calculateSalary(): float // Método sobreescreve o método da classe mãe
   {
     return parent::calculateSalary() + $this->calculateCommission();
   }
}
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

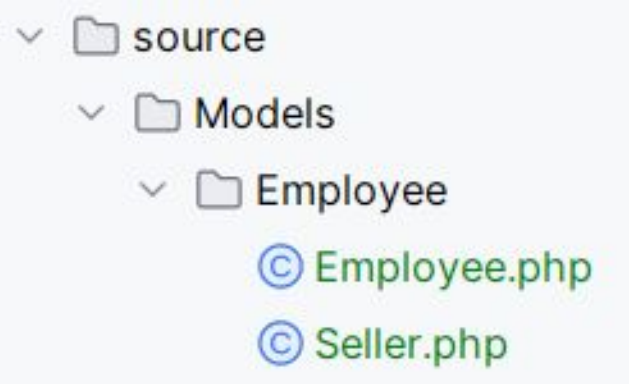
Orientação a Objetos em PHP

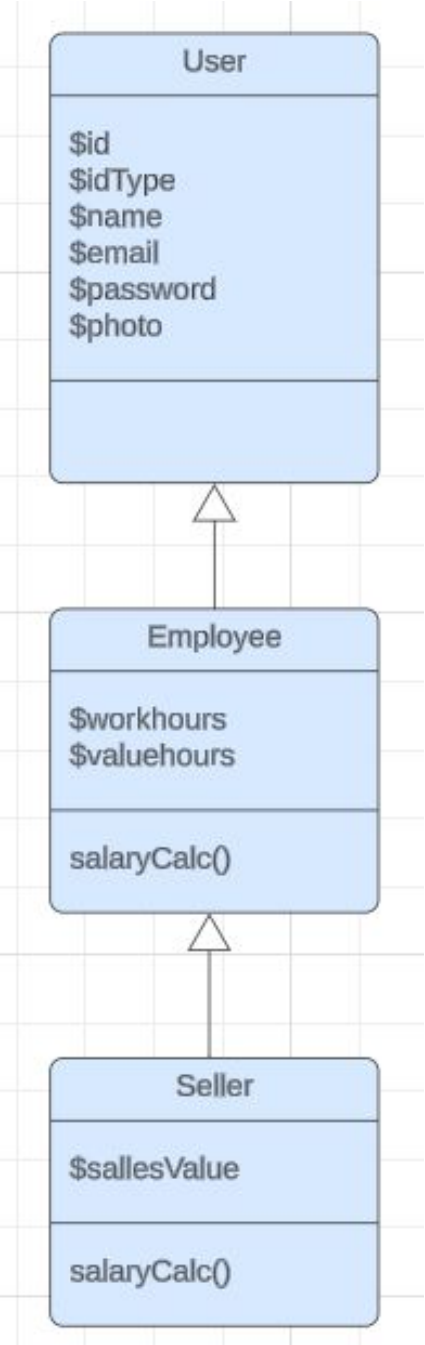
Polimorfismo

Crie uma nova namespace chamada Employee (source/Models/Company) para as classes que representam a seguinte realidade: em uma empresa é necessário manter os registros dos funcionários com as seguintes informaçõe: nome; e-mail; senha; foto (mesmos atributos da classe User)...

INF-3AM: https://github.com/sorfabiosantos/inf-3am-26/tree/main/exercicios/exercicio08
INF-3AT: https://github.com/sorfabiosantos/inf-3at-26/tree/main/exercicios/exercicio08

EDUCAÇÃO PÚBLICA 100% GRATUITA


source
Models
Employee
Employee.php
Seller.php


User
$id
$idType
$name
$email
$password
$photo
Employee
$workhours
$valuehours
salaryCalc()
Seller
$sallesValue
salaryCalc()

Orientação a Objetos em PHP

Associação

Na POO, a associação é um relacionamento entre classes de objetos. Ela permite que uma classe use objetos de outra classe para realizar operações.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Para gerenciar os dados cadastrais de uma empresa, crie uma Classe contendo o Nome da empresa e o Endereço. Crie uma classe para os dados do Endereço (Rua, Número e Complemento) que se associa à Empresa.

Orientação a Objetos em PHP

Associação

```php
<?php

namespace source\Models\Records;

class Company
{
   private $name;
   private $address;

   public function __construct($name, Address $address)
   {
       $this->name = $name;
       $this->address = $address;
   }

   public function getName()
   {
       return $this->name;
   }

   public function getAddress(): Address
   {
       return $this->address;
   }
}
```


EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Associação

```php
<?php

namespace source\Models\Records;

class Address
{
   private $street;
   private $number;
   private $complement;

   public function __construct($street, $number,
$complement)
   {
       $this->street = $street;
       $this->number = $number;
       $this->complement = $complement;
   }

   public function getStreet()
   {
       return $this->street;
   }

   public function getNumber()
   {
       return $this->number;
   }

   public function getComplement()
   {
       return $this->complement;
   }

   public function show() : string
   {
       return "{$this->street}, {$this->number}. {$this->complement}";
   }
}
```

07-associacao/index.php

```php
<?php

require __DIR__ . "/source/autoload.php";

$address = new \Source\Records\Address("General Balbão","324","");

$company = new \Source\Records\Company("IFSul - Câmpus Charqueadas", $address);

var_dump($company);

// alternativa

$companyABC = new \Source\Records\Company(
   "ABC - Advogados Associados",
   new \Source\Records\Address("Borges de Medeiros","1000","Conjunto 504")
);

var_dump($companyABC);

echo "<p>Empresa: {$companyABC->getName()}</p>
     <p>Endereço: {$companyABC->getAddress()->getStreet()}, {$companyABC->getAddress()->getNumber()} </p>
     <p>Complemento: {$companyABC->getAddress()->getComplement()}</p>";

echo "<p>Empresa: {$companyABC->getName()}</p>
     <p>Endereço: {$companyABC->getAddress()->show()}</p>";
```

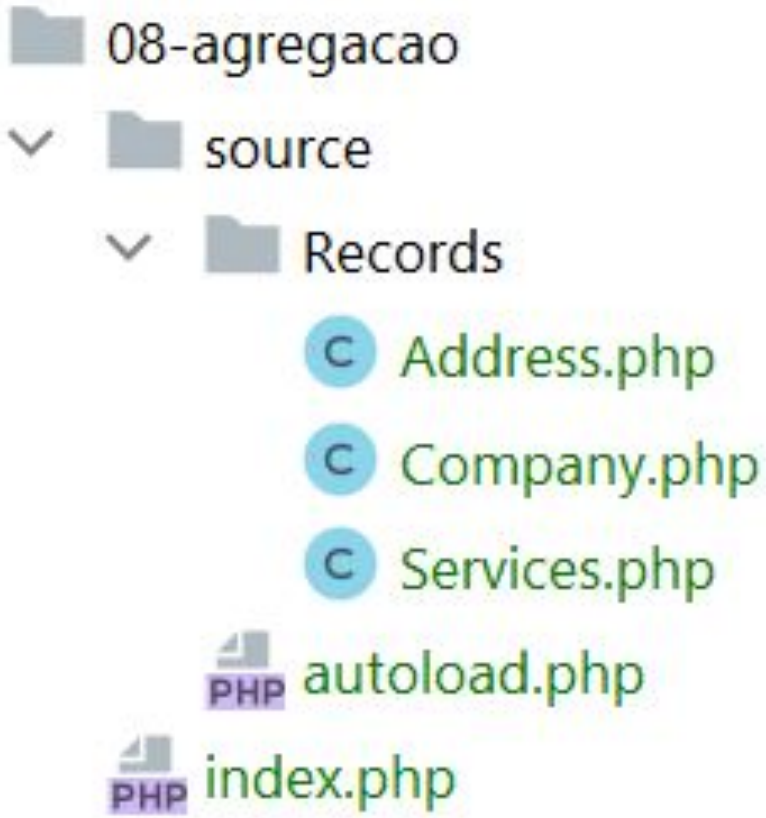
Agregação em POO é um mecanismo que permite reutilizar classes para compor outras classes. É um tipo de composição que representa um vínculo fraco entre duas classes.

EDUCAÇÃO PÚBLICA 100% GRATUITA

Seguindo o exemplo anterior, agora crie uma classe para gerenciar os Serviços prestados pela Empresa. Os serviços devem ser agregados à classe Empresa.

Orientação a Objetos em PHP

Agregação



EDUCAÇÃO PÚBLICA 100% GRATUITA

Orientação a Objetos em PHP

Agregação

```php
<?php

namespace Source\Records;

class Services
{
   private $name;
   private $price;
   /**
    * @param $name
    * @param $price
    */
   public function __construct($name, $price)
   {
       $this->name = $name;
       $this->price = $price;
   }

   public function getName()
   {
       return $this->name;
   }

   public function setName($name): void
   {
       $this->name = $name;
   }

   public function getPrice()
   {
       return $this->price;
   }

   public function setPrice($price): void
   {
       $this->price = $price;
   }
}
```

06-agregacao/index.php

```php
<?php

require __DIR__ . "/source/autoload.php";

$address = new \Source\Records\Address(
   "Felipe Camarão",
   "12323",
   "Sala 402"
);

$serv01 = new \Source\Records\Services("Inventário","10000");
$serv02 = new \Source\Records\Services("Separação","5000");

var_dump($address, $serv01, $serv02);
// Associação
$company = new \Source\Records\Company(
   "ABC - Advogados Associados",
   $address
);
```

06-agregacao/index.php

```
// Agregação
$company->addService($serv01);
$company->addService($serv02);
//ou de forma alternativa
$company->addService(new \Source\Records\Services(
   "Contratos de Compra e Venda",
   "500"
));

var_dump($company);

foreach ($company->getServices() as $service){
   echo "<p>{$service->getName()} - Valor {$service->getPrice()}</p>";
}
```

Orientação a Objetos em PHP

Abstração

É o pilar que simplifica sistemas complexos, focando apenas nos atributos e comportamentos essenciais de um objeto para o contexto atual, ignorando detalhes irrelevantes. Ela utiliza classes para modelar entidades do mundo real, permitindo criar representações genéricas e gerenciar a complexidade.

A abstração é frequentemente confundida com o encapsulamento, mas a abstração é sobre o que ocultar, enquanto o encapsulamento é sobre como ocultar esses detalhes.

Orientação a Objetos em PHP

Abstração

```
abstract class PaymentMethod
{
   protected ?float $amount;

   public function __construct(?float $amount = null)
   {
       $this->amount = $amount;
   }

   public function displayAmount(): string
   {
       return "Pagamento: R$ " . number_format($this->amount, 2, ',', '.');
   }
   // É obrigatória a implementação dos métodos abstratos pelas classes filhas
   abstract public function process();
   abstract public function redirect();
}
```

Orientação a Objetos em PHP

Abstração

```
class PixPayment extends PaymentMethod
{
   public function process(): string
   {
       return "Gerando o QR-code para R$ {$this->amount}... PIX realizado com sucesso!";
   }

   public function redirect(): string
   {
       return "Redirecionando para página do PIX";
   }

}

class CardPayment extends PaymentMethod
{
   public function process(): string
   {
       return "Conectando... Pagamento R$ {$this->amount} realizado com sucesso!";
   }

   public function redirect(): string
   {
       return "Redirecionando para página da Operadora do Cartão de Crédito";
   }
}
```

Professor Fábio Luís da Silva **Santos**

www.ifsul.edu.br
fabiosantos@ifsul.edu.br